

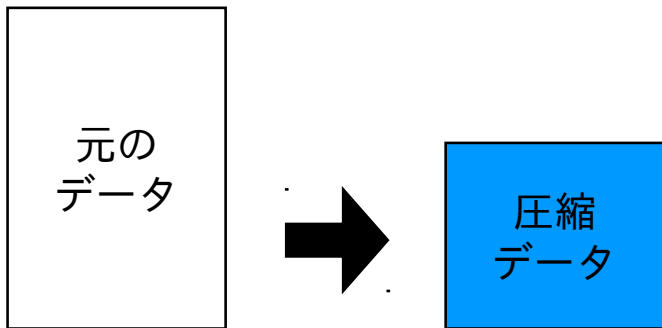
情報通信実験 2 講義資料 情報理論前半

はじめに

- ▶ 本実験では各種の演習を通じて情報理論、特にデータ圧縮に関連する代表的な手法を学ぶ
- ▶ サポート対象の言語はCとするが、その他の言語を利用してもよい
- ▶ 参考文献：
 - ▶ 植松友彦, 文章データ圧縮アルゴリズム入門, CQ 出版, 1994
 - ▶ 植松友彦, イラストで学ぶ情報理論の考え方, 講談社, 2012
 - ▶ T. M. Cover, J. A. Thomas, 山本博資, 古賀弘樹, 有村光晴, 岩本貢, 情報理論 -基礎と広がり-, 共立出版, 2012

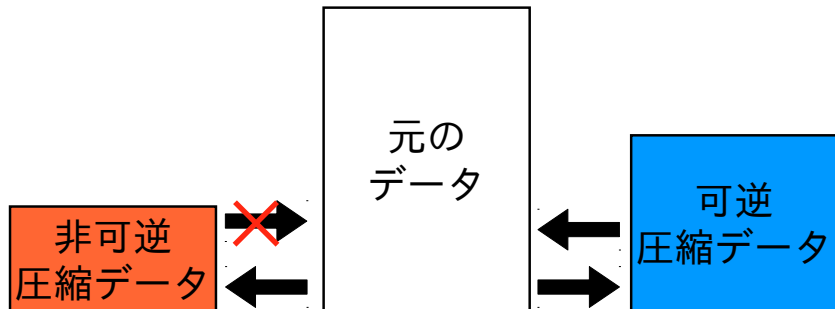
1. データ圧縮と符号化

データ圧縮とは



- ▶ データをサイズが小さくなるように別の形のデータに変換すること
- ▶ メモリの節約やデータ通信量の削減が可能

可逆圧縮と非可逆圧縮



▶ 可逆圧縮

- ▶ 圧縮データから元のデータに戻せる（可逆）
- ▶ データサイズは非可逆圧縮より大きい

▶ 非可逆圧縮

- ▶ 圧縮データから必ずしも元のデータに戻せるとは限らない（非可逆）
- ▶ データサイズは可逆圧縮より小さい

身の回りのデータ圧縮の規格

データ圧縮は色々なところで利用されている。

- ▶ ZIP（可逆圧縮）
- ▶ PNG（可逆圧縮）
- ▶ JPEG（非可逆圧縮）
- ▶ MP3（非可逆圧縮）
- ▶ HEVC（非可逆圧縮）
- ⋮

データ圧縮の実際

データ圧縮は符号化として扱われる。

符号化に関する「言葉」の定義

- ▶ 符号化：文章や文字列、プログラムなどに用いられる記号（情報）を0と1のビット列[†]によって表すこと
- ▶ 情報源アルファベット：出現する可能性のある記号の集合
- ▶ 符号語：記号を符号化した結果のビット列（圧縮データ）
- ▶ 符号語長：符号語の長さ

表: 符号語の例 (情報源アルファベット = $\{a, b, c\}$)

記号	符号語	符号語長
a	0	1
b	01	2
c	001	3

記号	符号語	符号語長
a	101	3
b	111	3
c	01	2

[†]一般にはビットに限らず、例えば0、1、2を使う3元の表現を用いてもよいが、簡単のためにビットを用いることにする。

ASCII コード

符号化の実例としては ASCII コードがある[†]。

アルファ ベット	ASCII コード	アルファ ベット	ASCII コード	アルファ ベット	ASCII コード
~	00100000	@	01000000	`	01100000
!	00100001	A	01000001	a	01100001
"	00100010	B	01000010	b	01100010
#	00100011	C	01000011	c	01100011
\$	00100100	D	01000100	d	01100100
%	00100101	E	01000101	e	01100101
&	00100110	F	01000110	f	01100110
'	00100111	G	01000111	g	01100111
(	00101000	H	01001000	h	01101000
)	00101001	I	01001001	i	01101001
*	00101010	J	01001010	j	01101010
+	00101011	K	01001011	k	01101011
,	00101100	L	01001100	l	01101100
-	00101101	M	01001101	m	01101101
.	00101110	N	01001110	n	01101110
/	00101111	O	01001111	o	01101111
0	00110000	P	01010000	p	01110000
1	00110001	Q	01010001	q	01110001
2	00110010	R	01010010	r	01110010
3	00110011	S	01010011	s	01110011
4	00110100	T	01010100	t	01110100
5	00110101	U	01010101	u	01110101
6	00110110	V	01010110	v	01110110
7	00110111	W	01000111	w	01110111
8	00111000	X	01011000	x	01111000
9	00111001	Y	01011001	y	01111001
:	00111010	Z	01011010	z	01111010
;	00111011	[	01011011	{	01111011
<	00111100	\	01011100		01111100
=	00111101	]	01011101	}	01111101
>	00111110	^	01011110	~	01111110
?	00111111	_	01011111		

[†]表は [植松友彦, 文章データ圧縮アルゴリズム入門, CQ 出版, 1994] からの引用

2. 語頭符号

語頭符号

- ▶ どの符号語も他の符号語の先頭部分と一致しない符号のこと
- ▶ 瞬時符号とも呼ばれる
- ▶ 符号語の列から一意に元の記号列を復号できる

表: 語頭符号の例

記号	符号語	符号語長
<i>a</i>	0	1
<i>b</i>	10	2
<i>c</i>	110	3

表: 語頭符号ではない例

記号	符号語	符号語長
<i>a</i>	0	1
<i>b</i>	01	2
<i>c</i>	010	3

語頭符号の例の場合：

$$bac \rightarrow 100110 \rightarrow bac$$

語頭符号ではない例の場合：

$$bac \rightarrow 010010 \rightarrow baba \text{ or } bac \text{ or } cba \text{ or } cc$$

符号の木：木による語頭符号の表現（つづく）

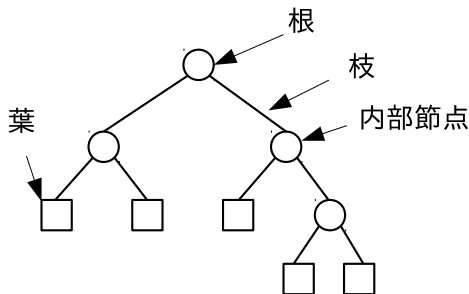


図: 木

- ▶ 枝：各線分
- ▶ 節点：枝の両端の点
- ▶ 根：一番上の節点
- ▶ 葉：節点から枝が一本も下に伸びていない節点
- ▶ 内部節点：葉でない節点

節点の1つ下の節点を「子」、1つ上の節点を「親」と呼ぶこともある。

符号の木：木による語頭符号の表現

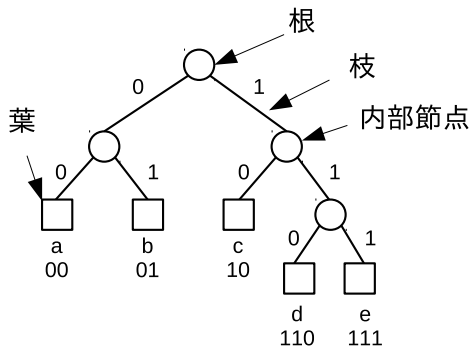


表: 符号の木に対応する符号

記号	符号語	符号語長
<i>a</i>	00	2
<i>b</i>	01	2
<i>c</i>	10	2
<i>d</i>	110	3
<i>e</i>	111	3

図: 符号の木

- ▶ 各枝にはラベル0か1が割り当てられている
- ▶ 葉に情報源アルファベットの1記号を対応させる
- ▶ 根から出発して、葉まで枝をたどって得られる0と1からなる系列がその葉の記号の符号語

クラフトの不等式

定理（語頭符号が存在するための符号語長分布に関する必要十分条件）

- ▶ 語頭符号が m 個の符号語をもち、それらの符号語長を $l_1, l_2, \dots, l_m$ とすると、

$$\sum_{i=1}^m 2^{-l_i} \leq 1 \quad (\text{クラフトの不等式})$$

を満たす

- ▶ この不等式を満たす正整数 $l_1, l_2, \dots, l_m$ が与えられたとき、これらを符号語長とする語頭符号を作ることができるし、逆にある語頭符号の符号語長は上の不等式を満たす。

語頭符号生成アルゴリズム

入力：クラフトの不等式を満たす正整数が昇順 $l_1 \leq l_2 \leq \dots \leq l_m$ で与えられている。

出力：記号 $x_1, \dots, x_m$ に対して、それぞれ符号語長 $l_1, \dots, l_m$ の符号語 $c_1, \dots, c_m$ を割り当てるアルゴリズムを与える。

- ① 深さ無限の2分木を用意して、 $i = 1, \dots, m$ に対して、以下を繰り返す。
- ② 木の深さ l_i の節点のうち、次を満たす一番左側の節点を選ぶ
 - ▶ 葉でない
 - ▶ 根から見てその節点より下に葉がない
- ③ 前項で選んだ節点より下の枝を全て削除してその節点を葉とする
- ④ 符号の木として見たときに、前項の葉に対応するビット列を記号 x_i に対応する符号語 c_i とする

符号構成の例

$l_1 = 1$ 、 $l_2 = 2$ 、 $l_3 = 2$ の場合：

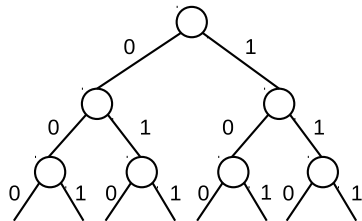


図: 初期状態の木

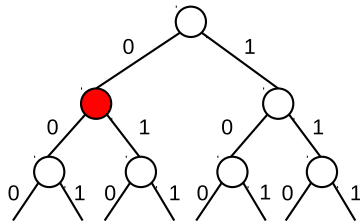


図: $l_1 = 1$ に対する節点の選択

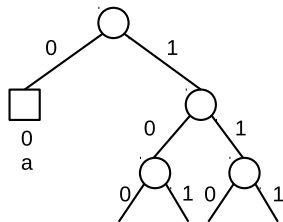


図: 節点を葉とする

符号構成の例

$l_1 = 1$ 、 $l_2 = 2$ 、 $l_3 = 2$ の場合：

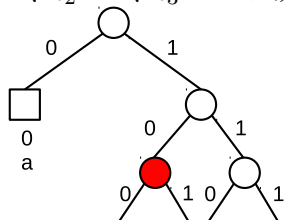


図: $l_2 = 2$ に対する節点の選択

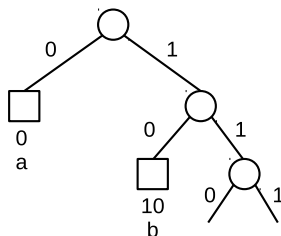


図: 節点を葉とする

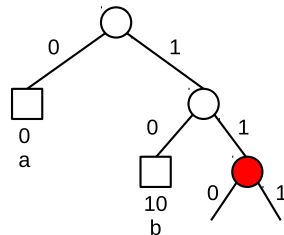


図: $l_3 = 2$ に対する節点の選択

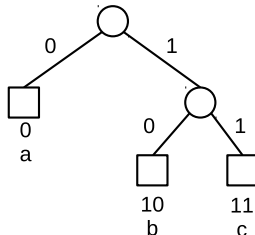


図: 節点を葉とする (符号の木の完成)

符号の木を使った復号化

与えられた符号語系列の先頭から順に、符号の木の根から枝をたどって得られる葉を出力することで記号列が復号できる。

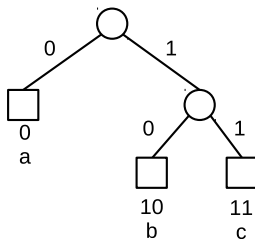


図: 符号の木

符号語系列 10001011 の復号化：

- ① 最初の 2 ビットの 10 まで根から枝をたどると記号 b の葉に到達する
- ② 残りの符号語系列 001011 に対して再び根から枝をたどる
- ③ 最初の 1 ビットの 0 をたどったところで記号 a の葉に到達する
- ④ 以下繰り返して記号列 baabc が得られる

平均符号語長とエントロピー

- ▶ 情報源アルファベット： $\{x_1, x_2, \dots, x_m\}$
- ▶ 記号 x_i の生起確率： $P(x_i)$
- ▶ x_i の符号語長： $l(x_i)$

$$\text{平均符号語長： } L \triangleq \sum_{i=1}^m P(x_i) l(x_i)$$

$$\text{エントロピー： } H(P) \triangleq - \sum_{i=1}^m P(x_i) \log_2 P(x_i)$$

平均符号語長の理論限界

どのような語頭符号を用いても、平均符号語長 L はエントロピー $H(P)$ を下回ることはいできない。すなわち、

$$H(P) \leq L$$

を満たす。

3. Shannon 符号とハフマン符号

Shannon 符号

平均符号語長がエントロピーに近くなるように、符号語長 $l(x_1), l(x_2), \dots, l(x_m)$ を選んで符号の木から語頭符号を作れば平均符号語長を小さくできると期待して、

$$l(x_i) = \lceil -\log P(x_i) \rceil \quad (\lceil x \rceil \text{ は } x \text{ の切り上げ})$$

とすると、

$$\text{クラフトの不等式: } \sum_{i=1}^m 2^{-l(x_i)} \leq \sum_{i=1}^m 2^{-\log \frac{1}{P(x_i)}} = \sum_{i=1}^m P(x_i) = 1$$

$$\text{平均符号語長: } \sum_{i=1}^m P(x_i) l(x_i) = \sum_{i=1}^m P(x_i) \lceil -\log P(x_i) \rceil \approx H(P)$$

Shannon 符号の作り方

上記の $l(x_i)$ を符号語長として語頭符号生成アルゴリズムから語頭符号を作成する。

ハフマン符号

- ▶ ハフマン木と呼ばれる符号の木を利用して符号化と復号化を行う
- ▶ 単一の符号の木から得られる最良の符号化方法
- ▶ $L \leq H(P) + 1$ が成り立つ

ハフマン木の作り方

- ① 各記号に対する葉を作り、それぞれの葉にその記号の生起確率[†]を書いておく
- ② 生起確率のもっとも小さい二つの葉に対し、新しい節点を一つ作り、その節点と2枚の葉を枝で結ぶ。この二つの枝の一方には0を、他方には1のラベルをつける。さらに、この新しい節点に、2枚の葉の確率の和を書き、この節点を新たな葉とみなす
- ③ 葉が1枚になるまで、2の手順を繰り返す

[†]記号列を符号化する際には、生起確率の代わりに、記号の記号列中の出現頻度を使う場合が多い。

ハフマン木の例

表: 記号と生起確率

記号	生起確率
<i>a</i>	$1/12$
<i>b</i>	$1/12$
<i>c</i>	$1/6$
<i>d</i>	$1/6$
<i>e</i>	$1/3$
—	$1/6$

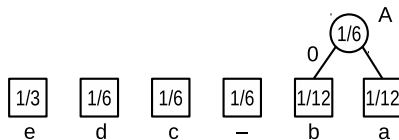


図: 生起確率のもっとも小さい二つの葉 *a* と *b* と新しい節点 *A* を枝で結ぶ。以降 *A* を葉とみなす。

ハフマン木の例

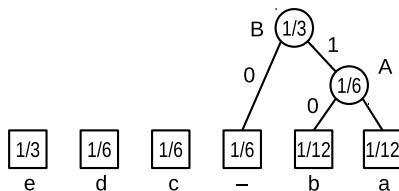


図: 生起確率のもっとも小さい二つの葉 “-” と A（葉とみなしていることに注意）と新しい節点 B を枝で結ぶ。以降、B を葉とみなす。

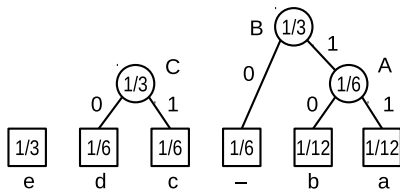


図: 生起確率のもっとも小さい二つの葉 d と c と新しい節点 C を枝で結ぶ。以降、C を葉とみなす。

ハフマン木の例

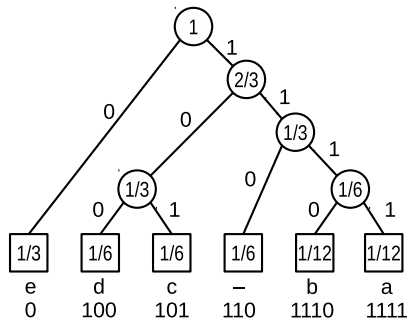


図: ハフマン木の完成

表: 符号語

記号	符号語
<i>a</i>	1111
<i>b</i>	1110
<i>c</i>	101
<i>d</i>	100
<i>e</i>	0
—	110

ヒープ

- ▶ 最小ヒープ[†]を利用して記号の生起確率を管理しておく、生起確率の最も小さい記号をソートすることなく選び出すことができる
- ▶ ヒープは木構造のデータ構造の一種であり次の性質を有する：
 - ▶ 木の各々の節点の値はその節点の子の値よりも小さいか等しい
 - ▶ すべての節点の値の中で根の値が常に最小
 - ▶ 葉は左詰めで配置される

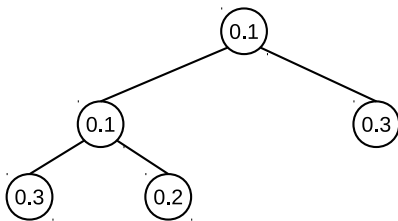


図: ヒープ

[†]これ以降、単にヒープと呼びます。

ヒープへの節点の追加

例：{0.1, 0.2, 0.3, 0.3} に 0.1 を追加したい。

- ① ヒープの最下段に左詰め[†]で節点を追加する
- ② 追加した節点の値とその親の値を比較し、正しい順序で並んでいるならば終了する
- ③ 順序が正しくない(親の値よりも子の値が小さい)ならば、親と追加した節点とを入れ替えて、2に戻る

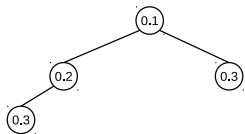


図: 初期状態

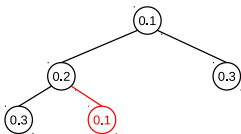


図: 節点の追加

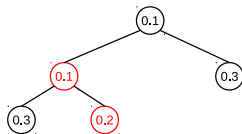


図: 節点の入れ替え

[†]幅優先探索で最初に見つかる追加可能な位置

ヒープからの根(最小値)の削除

例：{0.1, 0.1, 0.2, 0.3, 0.3} から、最小値 0.1 を削除したい。

- ① ヒープの根を最下段の最右の葉[†]で上書きし、最下段の最右の葉を削除する
- ② 新しい根の値とその子の値を比べて、子の方が小さければ子と親を入れ替える。ただし、子が2つある場合には、値の小さい方の子と入れ替える。同じ値なら、どちらでもいいが、左の子を選ぼう。
- ③ 2 の操作を、親の方が値が小さくなるか、葉に達するまで繰り返す

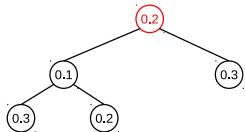


図: 根の上書き

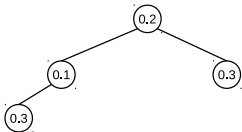


図: 葉の削除

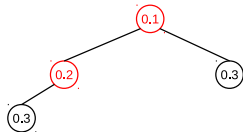


図: 節点の入れ替え

[†]幅優先探索で最後に見つかる葉